

MESERLIKE(메서라이크)

작성자 : 안승현

목차

1. 메서라이크 포트폴리오 소개
2. Cocos2d-x
3. 매니저 클래스
4. 월드맵
5. 인풋 매니저
6. 데이터 매니저
7. 콘텐츠 오브젝트
8. Cocos2-dx의 애니메이션 확장

9. 플레이어 컨트롤러
10. 몬스터 컨트롤러
11. 게임 매니저
12. 몬스터 웨이브
13. 피격
14. 스킬
15. 스킬 북
16. 깃허브 및 유튜브 링크

1. 메서라이크 포트폴리오 소개



- 사용 엔진 : Cocos2d-x
- 엔진 버전 : 3.17.2
- 사용 언어 : C++
- 작업 인원 : 1명
- 작업 영역 : 콘텐츠 제작, 디자인, 기획
- 장르 : 서바이벌 생존
- 소개 : 메이플 리소스를 활용한 뱀파이어 서바이벌
- 플랫폼 : PC
- 형상관리 : GitHub Desktop

2. COCOS2D-X



- Cocos2d는 대표적인 오픈 소스 2D 게임 엔진
- C++, JavaScript, Lua 언어를 지원하는 오픈소스 멀티 플랫폼 2D 게임 엔진
- 저사양 기기에서도 낮은 메모리 사용량과 빠른 속도로 2D 게임을 개발할 수 있게 해주는 강력한 프레임워크가 특징이다.

3. 매니저 클래스

```
class Managers
{
public:
    Managers();
    virtual ~Managers();

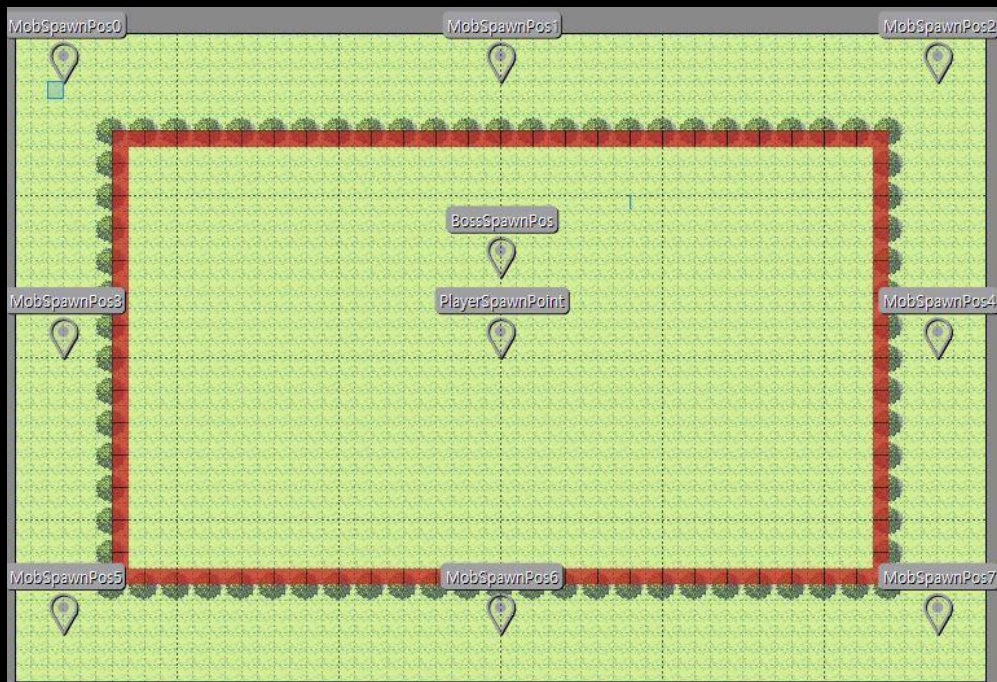
    static Managers& GetInstance();
private:
    static Managers instance;
```

- Cocos2d-x는 GUI 기반으로 설정할 수 없고 유니티에 비해 지원하는 라이브러리가 적어 직접 자주 사용하는 전역 클래스를 생성해 정의한다.
- 유일성을 만족하고 특정 기능을 수행하는 클래스 (사운드, 데이터, UI 등)들을 전역변수로 선언

소스 코드 : [Managers.h](#) / [Managers.cpp](#)



4. 월드맵



- 월드맵을 제작하기위해 외부 Tiled맵 사용
- 실제 컨텐츠가 플레이 되는 맵 을 구현 할려고 했으나 cocos2d-x GUI 기능을 지원하지 않기 때문에
- Tiled 외부 프로그램을 이용하여 타일 배치 및 스폰 위치 배치하여 월드를 구현

```
void MapManager::LoadMap(std::string path, cocos2d::Node* node, int layer, int tag)
{
    // 타일맵 불러오기
    tmap = TMXTiledMap::create(path);
    background = tmap->getLayer("Background");
    block = tmap->getLayer("Block");
    block->setVisible(false);
    // 타일맵에서 Objects라고 지정한 오브젝트 레이어의 객체들 가져오기
    objectPos = tmap->getObjectGroup("ObjectLayer1");
    // z-order를 -1로 하면 화면에 출력이 되지 않으니 0이상으로 설정한다.
    node->addChild(tmap, layer, tag);
}
```

```
cocos2d::Vec2 MapManager::GetObjectPos(std::string name)
{
    // 오브젝트 레이어에서 SpawnPoint라고 명명한 오브젝트를 가져오기
    ValueMap spawnPoint = objectPos->getObject(name);

    int x = spawnPoint["x"].asInt();
    int y = spawnPoint["y"].asInt();

    return Vec2(x, y);
}
```

- MapManager가 로드하고 싶은 맵을 매개변수로 받아 초기화 진행
- 초기화를 진행하면 Tiled에서 설정한 Tag 기반으로 플레이어, 몬스터 스폰 좌표를 가져와 스폰 할 수 있습니다.

소스코드 : [MapManager.h](#) / [MapManager.cpp](#)

5. 인풋 매니저

```
void InputManager::SetMoveInput(cocos2d::Point input)
{
    moveInput = input;
}

cocos2d::Point InputManager::GetMoveInput()
{
    return moveInput;
}
```

소스 코드 : [InputManager.h](#) / [InputManager.cpp](#)

- 유저 인풋을 받아 플레이어를 조작하기 위해서 구현
- 싱글톤인 Manager에서 접근 및 관리되며 UI에서 입력을 받아 변수에 인풋 값을 저장시킨다.
- 이러한 구현한 이유는 인풋 클래스는 다른 클래스를 알 필요 없고 필요한 클래스만 해당 클래스에 접근시키기 때문에 구현이 간단하다.

6. 데이터 매니저

```
DataManager::DataManager()  
{  
    LoadPlayerStatData();  
    LoadMonsterStatData();  
    LoadRoundSpawnData();  
    LoadSpriteData();  
    LoadMonsterData();  
    LoadPlayerData();  
    LoadRepeatSkillData();  
    LoadBuffSkillData();  
    LoadSequenceSkillData();  
    LoadPortionData();  
    LoadPlayerUseableSkillData();  
    LoadCreatureSoundData();  
    LoadFirstSkillSetData();  
}
```

- 전역 변수로 유일하는 존재하는 객체이며
- 게임 콘텐츠에 사용되는 여러 데이터를 효율적으로 관리하기 위해 구현
- 데이터 로드
 - LoadData 를 통해 JSON 데이터 로드
불러온 Json데이터를 Document 객체로 변환 및 반환
 - 반환된 Document 객체를 파싱하여 각 데이터 타입에 맞게 데이터 삽입

7. 콘텐츠 오브젝트

```
Dir GetDir();  
void SetDir(Dir dir);  
void FlipDir();  
  
AnimateEx* GetAnimateEx();  
  
Stat* GetStat();  
BaseController* GetBaseController();  
SkillBook* GetSkillBook();  
void StopHitAction();
```

소스 코드 : [Creature.h](#) / [Creature.cpp](#)

- 콘텐츠 오브젝트 대한 추상 클래스 정의
- 플레이어와, 몬스터 클래스는 비슷한 함수와 변수가 존재하기에 매번 클래스를 생성할때마다 코드를 작성하는 것은 비효율적이다.
- 부모 클래스인 Creature 클래스에서 공용으로 사용하는 기능을 구현 하거나 추후에 확장이 가능하도록 가상함수 선언
- 스탯, 애니메이션 등 여러 데이터를 관리

8. COCOS2D-X 애니메이션 확장

```
void Init(cocos2d::Node* owner, std::string name,
         cocos2d::Animation* defaultAnimate, bool roop = true, bool clear = true);

void SetOwner(cocos2d::Node* owner);

// 애니메이션을 넘겨주면 애니메이트로 변환되어 관리된다.
bool RegisterAnimation(std::string name,
                     cocos2d::Animation* animation, bool roop = true);

void PlayAnimation(std::string name);

void StopAnimation();

void Claer();
```

```
private:
    // 애니메이션을 실행할 대상
    cocos2d::Node* _owner;

    // 실행 중인 액션
    cocos2d::Action* animAction;

    // 등록된 애니메이트
    std::map<std::string, std::pair<bool, cocos2d::Animate*>> animMap;
```

- 기존의 Cocos-2dx 애니메이션은 한번 만들고 실행하는 기능밖에 없다.
- 커스텀 클래스를 생성하여 Cocos2d-x 애니메이트를 멤버변수로 가지고 이를 내부적으로 캐싱하여 재사용 가능한 컴포넌트 클래스
- 애니메이션을 애니메이트로 변환후 Map 자료 구조에 캐싱
- 필요시 키 값으로 애니메이트를 불러와 실행시키는 방법으로 구현

소스 코드 : [AnimateEx.h](#) / [AnimateEx.cpp](#)

9. 플레이어 컨트롤러

```
void PlayerController::Update(float deltaTime)
{
    switch (state)
    {
        case State::None:
            break;
        case State::Idle:
            UpdateIdle();
            break;
        case State::Move:
            UpdateMove();
            break;
        case State::Dead:
            break;
        default:
            break;
    }
}
```

- 플레이어의 직접 조작하기 위한 클래스
- BaseController를 상속받아 클래스 확장 및 정의
- FSM으로 행동 로직이 구현되어 있으며
InputManager의 멤버변수 값을 통해 행동로직
의 실행여부를 결정
- 현재 상태에 맞는 애니메이션 실행

소스 코드 : [PlayerController.h](#) / [PlayerController.cpp](#)

10. 몬스터 컨트롤러

```
void NormalMonsterController::Update(float deltaTime)
{
    switch (state)
    {
        case State::None:
            break;
        case State::Idle:
            UpdateIdle();
            break;
        case State::Move:
            UpdateMove();
            break;
        case State::Dead:
            break;
        default:
            break;
    }
}
```

- 몬스터 AI를 구현하기 위한 클래스
- BaseController를 상속받아 클래스 확장 및 정의
- FSM으로 행동 로직이 구현되어 있다.
- FSM으로 구현되어 있기에 다른 동작을 구현하고 싶으면 MonsterController 클래스를 상속받아 구현한다.

소스 코드 : [MonsterController.h](#) / [MonsterController.cpp](#)

11. 게임 매니저

```
Monster* SpawnMonster(MonsterType type);
Player* SpawnPlayer(int level);

void DeSpawnCreature(Creature* creature);

Player* GetPlayer();

void Clear();

int GetScore();
void SetScore(int value);

int GetRound();
void SetRound(int value);

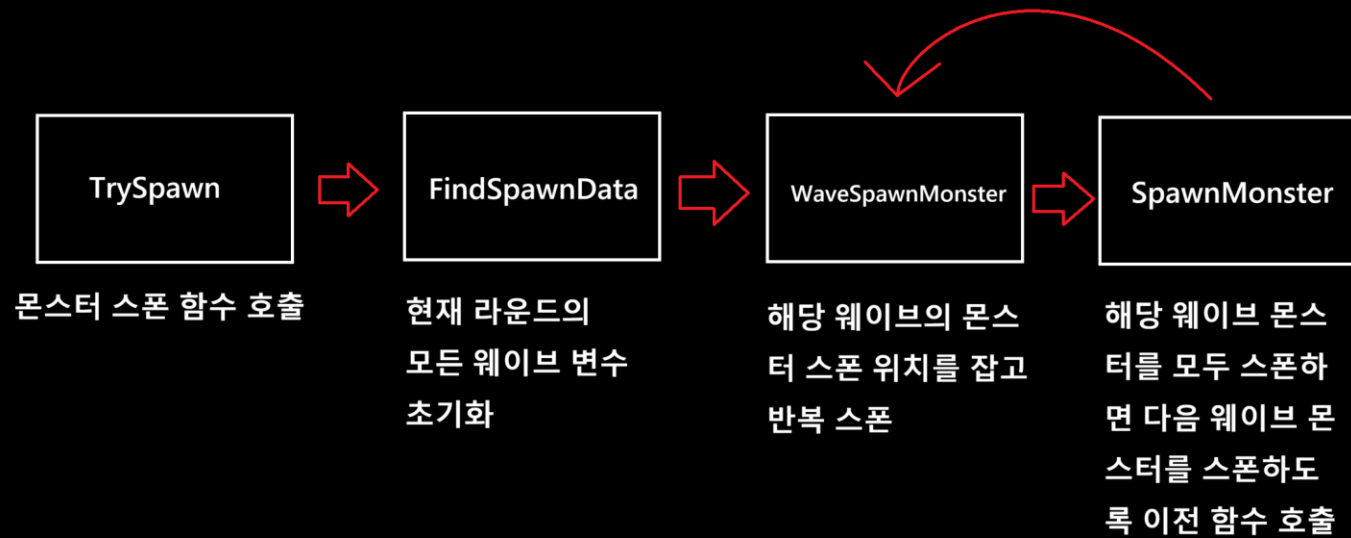
DelegateAction<int> roundDelegate;

std::vector<std::pair<int, SkillId>> GetRandomSkillId(int size = 3);
void SelectSkill(int index, SkillId skillId, SkillInfo skillInfo, SkillBase* skillBase);
```

- 게임 규칙 및 오브젝트를 관리하는 GameManager 클래스 생성
- 월드 콘텐츠에서 수행되는 정보를 쉽게 접근하기 위해
- 싱글톤 클래스로 구현했으며 해당 월드 콘텐츠에 대한 정보를 정의하는 방식으로 구현

소스 코드 : [GameManager.h](#) / [GameManager.cpp](#)

12. 몬스터 웨이브 구현



- 뱀서라이크 류 게임의 몬스터 스폰 방식을 따라 구현
- 웨이브 데이터 기반으로 SpawningPool 클래스가 웨이브에 있는 모든 종류를 스폰하고 모든 몬스터를 처치해야 다음 웨이브 작동 모든 웨이브가 작동되면 게임 종료

소스 코드 : [SpawningPool.h](#) / [SpawningPool.cpp](#)

몬스터 웨이브 데이터 구조

```
struct SpawnWaveArrayData
{
    int monsterId = 0;
    int spawnCount = 0;
};

struct SpawnArrayData
{
    float spawnWaveInterval = 0;
    std::vector<SpawnWaveArrayData> spawnWaveData;
};

struct RoundSpawnData
{
    int roundId = 0;
    float spawnInterval = 0;
    SpawnArrayData spawnData;
};
```

- RoundSpawnData
 - 해당 라운드에 스폰할 웨이브 리스트 정의
- SpawnArrayData
 - 해당 웨이브에서 스폰할 몬스터 리스트 정의
- SpawnWaveArrayData
 - 스폰할 몬스터와 수 정의

소스 코드 : [Data.h](#)

13. 피격

```
namespace cocos2d
{
    class Node;
}

class IHitable
{
public:
    virtual void OnAttacked(float damage) = 0;
    virtual void OnAttacked(cocos2d::Node* other, float damage) = 0;
};
```

```
class Player : public Creature, public IHitable
```

```
class NormalMonster : public Monster, public IHitable
```

```
class BossMonster : public Monster, public IHitable
```

소스 코드 : [Ihitable.h](#)

- 피격 인터페이스 구현
- 피격 대상 오브젝트인지 쉽게 판별하기 위해 구현
- 이유는 해당 클래스가 인터페이스를 상속받는지만 체크하면 되기 때문
- OnAttacked 순수 가상함수를 구현해 피격 처리를 하는 플레이어, 몬스터 클래스에서 상속받아 구현

14. 스킬

```
public:
    SkillBase(SkillType skillType, Creature* parent, int level = 1);
    virtual ~SkillBase();

    virtual bool SetLevel(int level);
    int GetLevel();

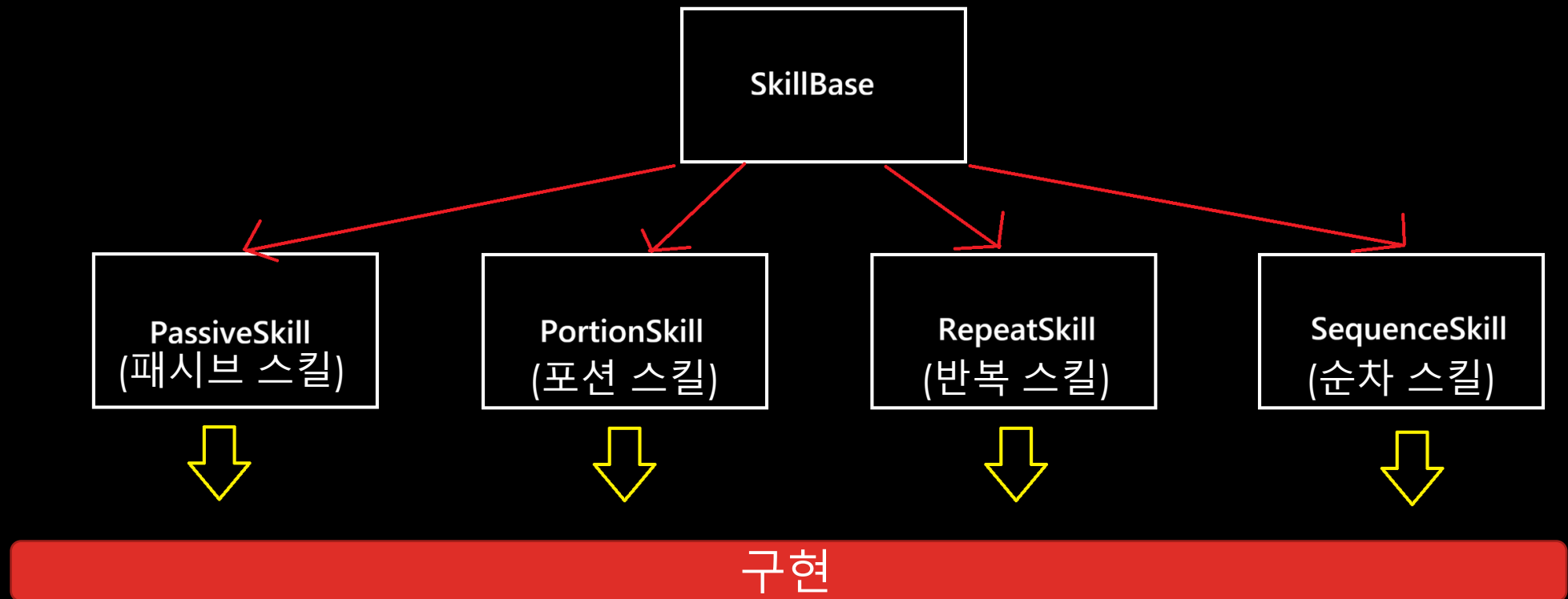
    virtual void ActivateSkill();

    void StartDestroy(float delaySeconds);
    void StopDestroy();
```

소스 코드 : [SkillBase.h](#) / [SkillBase.cpp](#)

- 뱀서라이크 스킬은 무한히 반복하는 스킬 구조를 가지기 때문에 해당 기능 구현
- 스킬을 정의하는 SkillBase 기반 기능 확장
 - 가상함수 SetLevel 함수에서 초기화
 - 가상함수 ActivateSkill 함수에서 기능 구현
- Skill 클래스 종류
 - 반복 스킬
 - 순차 스킬
 - 일회성 스킬
 - 지속 스킬

스킬 구조도



소스 코드 : [PasstiveSkill](#) / [PortionSkill](#) / [RepeatSkill](#) / [SequenceSkill](#)

15. 스킬 북

```
void SkillBook::AddSkillById(  
    SkillId skillId,  
    cocos2d::Vec2 position,  
    Creature* parent, int level)  
{  
    SkillBase* skillBase = SkillDefine::AddSkill(skillId, parent, level);  
  
    if (skillBase)  
    {  
        Skills.push_back(skillBase);  
  
        if (skillBase->GetSkillType() == SkillType::Repeat)  
            RepeatedSkills.insert({ skillId, skillBase });  
        else if (skillBase->GetSkillType() == SkillType::Sequence)  
            SequenceSkills.push_back(dynamic_cast<SequenceSkill*>(skillBase));  
        else if (skillBase->GetSkillType() == SkillType::Buff)  
            buffSkills.insert({ skillId, dynamic_cast<PassiveSkill*>(skillBase) });  
  
        parent->addChild(skillBase);  
    }  
}
```

- 구현한 스킬을 관리하기 위해 스킬 북 구현
- Creator 클래스에 SkillBook을 멤버변수 정의
- 스킬을 관리하는 SkillBook
 - 해당 오브젝트가 지니고 있는 스킬을 관리
 - 추가/삭제/조회 가능
- 스킬 등록은 SkillDefine 클래스에서 ID기반으로 데이터를 가져와 객체 반환

소스 코드 : [SkillBook.h](#) / [SkillBook.cpp](#) / [SkillDefine.h](#)

16. 깃허브 및 유튜브 링크



- Git HUB : <https://github.com/k660323/Meserlike>
- YouTube : <https://www.youtube.com/watch?v=zycwzQuWkkU>